Funkcje, słowniki, rekurencja

Python na maturę z informatyki · skrypt do pracy samodzielnej · 135 minut

Plan (orientacyjnie):

Część	Czas
Przygotowanie	10 min
Temat 1. Funkcje	25 min
Temat 2. Słowniki i zbiory	25 min
Temat 3. Comprehensions i sortowanie	25 min
Temat 4. Gotowe narzędzia z biblioteki standardowej	25 min
Temat 5. Rekurencja w Pythonie	20 min
Wysłanie plików	5 min

Zadanie dodatkowe na końcu skryptu robisz w domu (albo zaczynasz, jeśli zostanie czas).

Przygotowanie

1. Utwórz folder `zajecia2` i skopiuj do niego **8 plików z danymi**: `liczby.txt`, `uczniowie.txt`, `tekst.txt` (znasz je z poprzednich zajęć) oraz nowe: `oceny.txt`, `kolo_informatyczne.txt`, `kolo_matematyczne.txt`, `tajne1.txt`, `tajne2.txt`.
2. Programy zapisuj **w tym samym folderze** co pliki z danymi.
3. Każde ćwiczenie to **osobny plik** o nazwie według wzoru `tT_cwN.py`, np. `t3_cw2.py` to temat 3, ćwiczenie 2.
4. Sprawdź wersję Pythona poleceniem `python --version` (w terminalu). W temacie 4 używamy `math.lcm`, które wymaga Pythona **3.9 lub nowszego**. Jeśli masz starszy, daj znać w komentarzu w pliku i pomiń ten fragment.
5. Przy każdym ćwiczeniu jest **oczekiwany wynik**. Jeśli Twój program wypisuje dokładnie to samo, jest dobrze. Zwracaj uwagę nawet na spacje i dwukropki, bo na maturze też liczy się dokładny format.
6. Dane „z klawiatury" podaję po strzałce: *Wejście* → …. W `input()` **nie wpisuj tekstu zachęty**.
7. Czytaj uważnie i **uruchamiaj każdy przykład u siebie**. Dopiero zrozumiany przykład rób ćwiczenia.

Co oddajesz

Wszystkie pliki `.py` z ćwiczeń wysyłasz **pod koniec zajęć**, tak samo jak na poprzednich zajęciach. Nie zostawiaj pustych plików: **jeśli nie umiesz zrobić całego ćwiczenia, oddaj to, co masz**. Częściowe rozwiązanie jest cenne, bo pokazuje, gdzie jesteś. Zadanie dodatkowe możesz wysłać później.

Temat 1. Funkcje (ok. 25 min)

Teoria

Definicja i wywołanie. Funkcję definiujemy słowem `def`, a jej treść jest wcięta. Słowo `return` kończy funkcję i oddaje wynik. *Parametr* to nazwa w definicji, *argument* to wartość podana przy wywołaniu.

```
def kwadrat(x):
    return x * x

print(kwadrat(5))      # 25
y = kwadrat(3) + 1     # wynik funkcji można podstawić gdziekolwiek
print(y)               # 10
```

Brak `return` oznacza wynik `None`. Typowy błąd to funkcja, która wypisuje zamiast zwracać:

```
def zle(x):
    print(x * 2)       # wypisuje, ale niczego nie zwraca

def dobrze(x):
    return x * 2

a = zle(4)             # wypisze 8, a = None
b = dobrze(4)          # b = 8
print(a, b)            # None 8
```

Zasada: funkcja **zwraca** wynik, a wypisuje go dopiero kod, który ją wywołał.

Zwracanie kilku wartości. Funkcja może zwrócić kilka wartości po przecinku (to krotka). Odbierasz je przez rozpakowanie, znane z tematu o listach:

```
def min_max(l):
    return min(l), max(l)

a, b = min_max([4, 9, 2])
print(a, b)            # 2 9
```

Argumenty domyślne i nazwane. Parametr może mieć wartość domyślną, używaną, gdy nie podasz argumentu. Parametry z wartością domyślną muszą być na końcu. Argumenty można też podawać z nazwy:

```
def potega(a, n=2):
    return a ** n

print(potega(5))         # 25   (n = 2)
print(potega(2, 10))     # 1024
print(potega(n=3, a=2))  # 8    (kolejność nie ma znaczenia przy podawaniu z
nazwy)
```

Zasięg zmiennych. Zmienna utworzona w funkcji istnieje tylko w niej (jest *lokalna*):

```
x = 10
def f():
    x = 5          # NOWA zmienna lokalna, nie zmienia x z zewnątrz
    return x

print(f(), x)      # 5 10
```

Funkcja może odczytać zmienną z zewnątrz, ale żeby ją zmienić, potrzebuje słowa `global`. **Unikaj `global`**, bo kod staje się trudny do śledzenia. Dane przekazuj parametrami, a wynik oddawaj przez `return`.

Lista przekazana do funkcji nie jest kopiowana. Działa tu ta sama pułapka co przy `b = a`: zmiana wewnątrz funkcji zmienia oryginał.

```
def dodaj(l):
    l.append(99)

a = [1, 2]
dodaj(a)
print(a)               # [1, 2, 99]
```

Dobre nawyki. Jedna funkcja robi jedną rzecz. Testuj ją od razu na prostym przykładzie. Na maturze funkcje oszczędzają pisanie, np. jedna funkcja `wczytaj(nazwa)` może obsłużyć kilka plików.

Ćwiczenia

Ćwiczenie 1 ★ *(plik `t1_cw1.py`)* Napisz funkcje `srednia(l)` i `ile_parzystych(l)`. Obie mają **zwracać** wynik (nie wypisywać). Wywołaj je dla listy `dane = [4, 8, 15, 16, 23, 42]` i wypisz wyniki.

Oczekiwany wynik:

```
Średnia: 18.00
Parzystych: 4
```

Ćwiczenie 2 ★★ *(plik `t1_cw2.py`)* Napisz funkcję `wczytaj(nazwa)`, która czyta plik z jedną liczbą w linii i zwraca listę liczb całkowitych. Napisz też funkcję `min_max(l)`, która **bez użycia wbudowanych `min` i `max`** (pętlą) zwraca parę: najmniejszy i największy element. Wczytaj `liczby.txt` i wypisz najmniejszą, największą liczbę oraz rozstęp (różnicę między nimi).

Oczekiwany wynik:

```
Najmniejsza: 3
Największa: 90
Rozstęp: 87
```

Ćwiczenie 3 ★★★ *(plik `t1_cw3.py`)* Liczba doskonała jest równa sumie swoich dzielników właściwych (czyli dzielników mniejszych od niej samej), np. 6 = 1 + 2 + 3. Napisz funkcję `dzielniki(n)` zwracającą listę dzielników właściwych, funkcję `doskonala(n)` zwracającą `True` lub `False` oraz funkcję `potega(a, n=2)` z argumentem domyślnym. Wypisz wszystkie liczby doskonałe od 2 do 1000 w jednej linii (`print(*lista)`), a potem `potega(5)` i `potega(2, 10)` w jednej linii.

Oczekiwany wynik:

```
Liczby doskonałe: 6 28 496
25 1024
```

Temat 2. Słowniki i zbiory (ok. 25 min)

Teoria

Słownik (`dict`) przechowuje pary *klucz → wartość* i pozwala bardzo szybko znaleźć wartość po kluczu. Kluczem może być liczba, napis lub krotka (nie lista).

```
oceny = {"Anna": 5, "Jan": 3}
oceny["Zofia"] = 4            # dodanie nowej pary (lub zmiana istniejącej)
print(oceny["Anna"])          # 5
print("Jan" in oceny)         # True  (pytamy o KLUCZ)
print(len(oceny))             # 3
del oceny["Jan"]              # usunięcie pary
print(oceny.get("Piotr"))     # None  (brak klucza, bez błędu)
print(oceny.get("Piotr", 0))  # 0     (własna wartość domyślna)
print(oceny["Piotr"])         # KeyError! zwykły dostęp do brakującego klucza
kończy się błędem
```

Przechodzenie po słowniku:

```
for k in oceny:               # po kluczach
    print(k)
for k, v in oceny.items():    # po parach (klucz, wartość)
    print(k, v)
print(list(oceny.values()))   # same wartości
```

Wzorzec 1: liczenie wystąpień. Bardzo częsty na maturze. `get(klucz, 0)` daje 0, gdy klucza jeszcze nie ma:

```
licznik = {}
for z in "banan":
    licznik[z] = licznik.get(z, 0) + 1
print(licznik)                # {'b': 1, 'a': 2, 'n': 2}
```

Wzorzec 2: grupowanie. `setdefault(klucz, wartość)` wstawia wartość, jeśli klucza brak, i w każdym przypadku zwraca to, co jest pod kluczem. Dzięki temu można od razu dopisywać do listy:

```
grupy = {}
for imie, klasa in [("Anna", "3A"), ("Jan", "3B"), ("Ewa", "3A")]:
    grupy.setdefault(klasa, []).append(imie)
print(grupy)                  # {'3A': ['Anna', 'Ewa'], '3B': ['Jan']}
```

Słownik zapamiętuje kolejność dodawania. Żeby wypisać go w porządku alfabetycznym, przejdź po `sorted(slownik)`.

Zbiór (`set`) to kolekcja **bez duplikatów i bez kolejności**. Sprawdzenie, czy element w nim jest (`in`), jest bardzo szybkie.

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b)                    # {3, 4}           część wspólna
print(a | b)                    # {1, 2, 3, 4, 5} suma
print(a - b)                    # {1, 2}          tylko w a
print(a ^ b)                    # {1, 2, 5}      w dokładnie jednym
print(3 in a)                   # True
a.add(9)                        # dodaj element
a.discard(1)                    # usuń element (bez błędu, gdy go nie ma)
print(set([1, 1, 2, 2, 3]))     # {1, 2, 3}  usuwanie duplikatów z listy
```

Uwaga: **pusty zbiór** tworzysz przez `set()`, a nie `{}` (to jest pusty słownik). Ponieważ zbiór nie ma kolejności, do wypisywania używaj `sorted(zbior)`.

Ćwiczenia

Ćwiczenie 1 ★ *(plik `t2_cw1.py`)* Wczytaj słowo i policz, ile razy występuje każda litera, używając słownika i wzorca liczenia z teorii. Wypisz wyniki w porządku alfabetycznym liter, w postaci `litera: ile`.

Wejście → `mississippi`. Oczekiwany wynik:

```
i: 4
m: 1
p: 2
s: 4
```

Ćwiczenie 2 ★★ *(plik `t2_cw2.py`)* Plik `oceny.txt` zawiera w każdej linii imię i ocenę oddzielone spacją (to samo imię występuje w wielu liniach). Zbierz oceny każdej osoby do słownika *imię → lista ocen* (wzorzec grupowania). Dla każdej osoby, w kolejności alfabetycznej, wypisz liczbę ocen i średnią z dwoma miejscami po przecinku.

Oczekiwany wynik:

```
Anna: ocen 4, średnia 4.25
Jan: ocen 3, średnia 3.33
Maria: ocen 2, średnia 4.50
Piotr: ocen 3, średnia 2.67
Zofia: ocen 3, średnia 4.67
```

Ćwiczenie 3 ★★★ *(plik `t2_cw3.py`)* Pliki `kolo_informatyczne.txt` i `kolo_matematyczne.txt` zawierają imiona uczniów zapisanych do kół (jedno imię w linii). Wczytaj każdy plik do **zbioru** (napisz do tego jedną funkcję `wczytaj(nazwa)`, która zwraca

zbiór). Wypisz osoby zapisane do obu kół, tylko do informatycznego, tylko do matematycznego (każdą grupę posortowaną i oddzieloną przecinkami, użyj `", ".join(...)`) oraz liczbę wszystkich różnych osób.

Oczekiwany wynik:

```
W obu kołach: Ewa, Jan, Piotr
Tylko informatyczne: Anna, Kamil, Tomasz, Zofia
Tylko matematyczne: Katarzyna, Marek, Maria
Razem różnych osób: 10
```

Temat 3. Comprehensions i sortowanie (ok. 25 min)

Teoria

List comprehension to skrócony zapis pętli, która buduje listę. Schemat: `[wyrażenie for x in sekwencja if warunek]`, gdzie `if` jest opcjonalne.

```
kw = [x * x for x in range(1, 6)]             # [1, 4, 9, 16, 25]
par = [x for x in range(10) if x % 2 == 0]    # [0, 2, 4, 6, 8]
dl = [len(s) for s in ["ala", "kota"]]         # [3, 4]
```

Pierwszy zapis robi dokładnie to samo co ta pętla:

```
kw = []
for x in range(1, 6):
    kw.append(x * x)
```

Podobnie działają **słownik** i **zbiór** zbudowane w jednym wyrażeniu, a także **wyrażenie w nawiasach okrągłych podane do `sum`, `max` itp.**:

```
print({x: x * x for x in range(4)})           # {0: 0, 1: 1, 2: 4, 3: 9}
print(sum(int(c) for c in "123"))              # 6  (suma cyfr)
```

Plik z liczbami wczytasz w jednej linii, np. wewnątrz `with`:

`liczby = [int(linia) for linia in f]`.

Wyjaśnienie wzorca z tematu o listach. Zapis `[[0] * 3 for _ in range(2)]` tworzy dwa **osobne** wiersze, bo wyrażenie `[0] * 3` jest wykonywane od nowa przy każdym obrocie. Nazwa `_` oznacza „zmienna, której nie potrzebuję".

Sortowanie. `sorted(kolekcja)` zwraca nową posortowaną listę (oryginał bez zmian). Parametr `reverse=True` odwraca kolejność. Parametr `key` pozwala wskazać, **według czego** sortować:

```
slowa = ["kot", "ala", "pies", "ma"]
print(sorted(slowa, key=len))          # ['ma', 'kot', 'ala', 'pies']  (wg
długości)

pary = [("Anna", 87), ("Jan", 64), ("Zofia", 92)]
print(sorted(pary, key=lambda p: p[1]))                 # [('Jan', 64),
('Anna', 87), ('Zofia', 92)]
print(sorted(pary, key=lambda p: p[1], reverse=True))   # [('Zofia', 92),
('Anna', 87), ('Jan', 64)]
```

`lambda p: p[1]` to krótka, bezimienna funkcja: dostaje `p` i zwraca `p[1]`. Tutaj mówi „sortuj według drugiego elementu pary".

Sortowanie po kilku kryteriach. Jako klucz podaj krotkę. Dla liczb minus przed wartością odwraca kolejność tego kryterium:

```
# malejąco po punktach, a przy remisie rosnąco po imieniu
sorted(pary, key=lambda p: (-p[1], p[0]))
```

Stabilność. Elementy o równym kluczu zachowują wzajemną kolejność z oryginału (zobacz `kot` przed `ala` wyżej).

`min` i `max` też przyjmują `key`:

```
print(max(pary, key=lambda p: p[1]))   # ('Zofia', 92)
print(min(slowa, key=len))             # ma
```

Sortowanie słownika po wartościach:

```
licznik = {"a": 5, "b": 2, "c": 7}
print(sorted(licznik.items(), key=lambda p: p[1], reverse=True))   # [('c',
7), ('a', 5), ('b', 2)]
```

Uwaga: napisy są porównywane według kodów znaków, więc polskie litery `Ć`, `Ł`, `Ż` trafiają **za** `Z`. Przy tych ćwiczeniach nie ma to znaczenia, ale warto pamiętać.

Ćwiczenia

Ćwiczenie 1 ★ *(plik `t3_cw1.py`)* Wczytaj `liczby.txt` do listy (comprehension). Używając **tylko comprehensions**, wypisz: (a) listę liczb parzystych, (b) listę kwadratów liczb większych od 50, (c) sumę tych kwadratów, (d) słownik, w którym kluczem jest liczba mniejsza od 20, a wartością suma jej cyfr.

Oczekiwany wynik:

```
Parzyste: [12, 88, 56, 34, 90, 18]
Kwadraty liczb > 50: [7744, 3136, 4489, 8100, 5329]
Suma kwadratów: 28798
Sumy cyfr: {12: 3, 7: 7, 3: 3, 9: 9, 5: 5, 18: 9}
```

Ćwiczenie 2 ★★ *(plik `t3_cw2.py`)* Wczytaj `uczniowie.txt` do listy krotek `(imię, nazwisko, klasa, punkty)` (punkty jako liczba). (a) Wypisz **ranking pięciu najlepszych** osób: malejąco po punktach, przy remisie rosnąco po nazwisku, w postaci `numer. Imię Nazwisko - punkty`. (b) Wypisz wszystkich uczniów posortowanych rosnąco po klasie, a w obrębie klasy malejąco po punktach, w postaci `klasa Imię Nazwisko punkty`.

Oczekiwany wynik:

```
Ranking:
1. Zofia Ćwik - 92
2. Anna Kowalska - 87
3. Ewa Zielińska - 83
4. Maria Dąbrowska - 78
5. Łukasz Żak - 71
Wg klas:
3A Anna Kowalska 87
3A Maria Dąbrowska 78
3A Łukasz Żak 71
3A Kamil Szymański 47
3B Ewa Zielińska 83
3B Jan Nowak 64
3B Piotr Wiśniewski 55
3C Zofia Ćwik 92
3C Katarzyna Wójcik 70
3C Tomasz Lewandowski 69
```

Ćwiczenie 3 ★★★ *(plik `t3_cw3.py`)* Wczytaj słowa z `tekst.txt` (małe litery, bez znaków `.,;:!?`, tak jak w zadaniu dodatkowym z poprzednich zajęć). Policz wystąpienia każdego słowa w słowniku. Wypisz **5 najczęstszych słów** (malejąco po liczbie wystąpień, przy remisie alfabetycznie) w postaci `słowo: ile`. Potem wypisz **3 najdłuższe różne słowa** (malejąco po długości, przy remisie alfabetycznie), oddzielone przecinkami. Podpowiedź: różne słowa dostaniesz przez `set(...)`.

Oczekiwany wynik:

```
to: 3
kajak: 2
na: 2
a: 1
ala: 1
Najdłuższe: programowanie, przetwarzaniu, nieszczęście
```

Temat 4. Gotowe narzędzia z biblioteki standardowej (ok. 25 min)

Teoria

Python ma wiele gotowych narzędzi, które trzeba tylko **zaimportować**. Importy piszesz na początku programu: `import math` (potem `math.gcd(...)`) albo `from collections import Counter` (potem samo `Counter(...)`).

`Counter`: liczenie wystąpień bez własnej pętli.

```
from collections import Counter

c = Counter("abrakadabra")
print(c["a"])              # 5
print(c["z"])              # 0   (brak elementu = 0, bez błędu)
print(c.most_common(2))    # [('a', 5), ('b', 2)]  najczęstsze, malejąco
```

Przy remisie `most_common` zachowuje kolejność pierwszego pojawienia się elementów. `Counter` działa też na listach.

`defaultdict`: słownik, który sam tworzy brakujące wartości, więc nie potrzeba `get` ani `setdefault`.

```
from collections import defaultdict

g = defaultdict(list)       # brakujący klucz = pusta lista
g["3A"].append("Anna")
licznik = defaultdict(int)  # brakujący klucz = 0
licznik["x"] += 1
```

`deque`: kolejka dwustronna. Dodawanie i zdejmowanie z **obu końców** jest szybkie (w liście `pop(0)` jest wolne). Przyda się przy kolejkach i przeszukiwaniu grafów (BFS).

```
from collections import deque

d = deque([1, 2, 3])
d.append(4)                 # deque([1, 2, 3, 4])
d.appendleft(0)             # deque([0, 1, 2, 3, 4])
print(d.popleft())          # 0   (zdjęcie z początku)
d.rotate(1)                 # obrót w prawo: deque([4, 1, 2, 3])
d.rotate(-1)                # obrót w lewo:  deque([1, 2, 3, 4])
```

`itertools`: generowanie konfiguracji kombinatorycznych. Różnica: w `permutations` liczy się **kolejność**, w `combinations` nie, a `product` pozwala **powtarzać** elementy.

Uwaga: `Permutations` nie zawsze oznacza permutacje w ujęciu matematycznym:

`permutations(elementy)` - permutacje (wszystkie elementy, istotna kolejność)

`permutations(elementy, n)` - wariacje bez powtórzeń (n elementów, istotna kolejność)

`combinations(elementy, n)` - kombinacje (n elementów, nieistotna kolejność)

`product(elementy, inne_elementy)` - iloczyn kartezjański (wszystkie możliwe zestawienia elementów)

`product(elementy, repeat=2)` - wariacje z powtórzeniami (szczególny przypadek iloczynu kartezjańskiego, gdzie dwa zbiory są identyczne)

```
from itertools import permutations, combinations, product

print(list(permutations([1, 2, 3], 2)))   # [(1, 2), (1, 3), (2, 1), (2, 3),
(3, 1), (3, 2)]
print(list(combinations([1, 2, 3], 2)))   # [(1, 2), (1, 3), (2, 3)]
print(list(product("ab", repeat=2)))      # [('a', 'a'), ('a', 'b'), ('b',
'a'), ('b', 'b')]
```

Każdy element takiego wyniku to krotka. Do sklejenia liter w napis użyj `"".join(krotka)`.

`math`: gotowe funkcje matematyczne.

```
import math

print(math.gcd(12, 18))      # 6    największy wspólny dzielnik (NWD)
print(math.lcm(4, 6))        # 12   najmniejsza wspólna wielokrotność (NWW)
print(math.isqrt(50))        # 7    całkowity pierwiastek (część całkowita)
print(math.sqrt(16))         # 4.0  zwykły pierwiastek (liczba rzeczywista)
print(math.factorial(5))     # 120
print(math.floor(2.7), math.ceil(2.1))   # 2 3   zaokrąglenie w dół i w górę
print(math.inf > 10 ** 100)  # True  "nieskończoność", przydatna jako
startowe minimum
```

W Pythonie 3.9+ `gcd` i `lcm` przyjmują dowolnie wiele argumentów, np. `math.gcd(*lista)`.

`bisect`: szybkie wyszukiwanie w **posortowanej** liście (wyszukiwanie binarne, czas O(log n)).

```
from bisect import bisect_left, bisect_right

a = [10, 20, 20, 30]
print(bisect_left(a, 20))        # 1   indeks pierwszej 20
print(bisect_right(a, 20))       # 3   indeks tuż za ostatnią 20
print(bisect_right(a, 20) - bisect_left(a, 20))   # 2   ile jest dwudziestek
print(bisect_left(a, 25))        # 3   gdzie wstawić 25, żeby lista została
posortowana
```

Pamiętaj, że `bisect_left(a, x)` to liczba elementów **mniejszych** od `x`, a `bisect_right(a, x)` to liczba elementów **nie większych** od `x`.

Ćwiczenia

Ćwiczenie 1 ★ *(plik `t4_cw1.py`)* Wczytaj słowo. Używając `Counter`, wypisz trzy najczęstsze litery w postaci `litera: ile` oraz liczbę różnych liter.

Wejście → `abrakadabra`. Oczekiwany wynik:

```
a: 5
b: 2
r: 2
Różnych liter: 5
```

Ćwiczenie 2 ★★ *(plik `t4_cw2.py`)* Używając `itertools`, wypisz: (a) wszystkie permutacje liter `abc` jako napisy w jednej linii, (b) liczbę wszystkich 3-elementowych kombinacji zbioru liczb od 1 do 5, (c) wszystkie napisy długości 3 złożone z cyfr `0` i `1` w

jednej linii, (d) liczbę trójek `a < b < c` z liczb od 1 do 20, których suma wynosi 30 (podpowiedź: `combinations` i pętla z licznikiem).

Oczekiwany wynik:

```
abc acb bac bca cab cba
Kombinacji: 10
000 001 010 011 100 101 110 111
Trójek: 45
```

Ćwiczenie 3 ★★★ *(plik `t4_cw3.py`)* Trzy krótkie części w jednym pliku.

(a) Dla listy `[84, 126, 210]` wypisz NWD i NWW wszystkich liczb naraz (`math.gcd(*l)`, `math.lcm(*l)`).

(b) Dla posortowanej listy `punkty = [12, 25, 33, 47, 58, 61, 79, 85, 90, 97]` i każdego `q` z `(60, 85, 100)` wypisz, ile elementów jest nie większych od `q` (`bisect_right`) i ile mniejszych od `q` (`bisect_left`).

(c) **Zadanie Józefa Flawiusza:** siedem osób ponumerowanych od 1 do 7 stoi w kółku. Zaczynając od osoby 1, odliczamy co trzecią osobę i ta odpada, po czym liczymy dalej od następnej. Wypisz kolejność odpadania i numer osoby, która zostanie. Podpowiedź: weź `deque(range(1, 8))`, w każdym kroku wykonaj `d.rotate(-2)` (dwie osoby przechodzą na koniec kolejki), a trzecią zdejmij przez `popleft()`. Powtarzaj, dopóki w kolejce jest więcej niż jedna osoba.

Oczekiwany wynik:

```
NWD: 42
NWW: 1260
60: nie większych 5, mniejszych 5
85: nie większych 8, mniejszych 7
100: nie większych 10, mniejszych 10
Kolejność odpadania: 3 6 2 7 5 1
Zostaje: 4
```

Temat 5. Rekurencja w Pythonie (ok. 20 min)

Teoria

Rekurencja to sytuacja, w której funkcja **wywołuje samą siebie** dla mniejszego problemu. Każda poprawna funkcja rekurencyjna ma dwie części: **warunek stopu** (przypadek tak prosty, że znamy wynik od razu) oraz **krok rekurencyjny** (sprowadzenie problemu do mniejszego).

```
def silnia(n):
    if n <= 1:                  # warunek stopu
        return 1
    return n * silnia(n - 1)    # krok: silnia(n) = n * silnia(n - 1)

print(silnia(5))                # 120
```

Jak to działa dla `silnia(3)`: wywołanie `silnia(3)` czeka na wynik `silnia(2)`, które czeka na `silnia(1)`. To ostatnie zwraca 1, więc `silnia(2)` zwraca `2 * 1`, a `silnia(3)` zwraca `3 * 2`, czyli 6. Wywołania czekające na wynik leżą na *stosie wywołań*, jedno na drugim.

Brak warunku stopu oznacza wywołania bez końca. Python przerywa je po ok. 1000 poziomach błędem `RecursionError`:

```
def zle(n):
    return zle(n + 1)

zle(0)    # RecursionError: maximum recursion depth exceeded
```

Limit można zwiększyć (`import sys` i `sys.setrecursionlimit(10000)`), ale **ostrożnie**: zbyt głęboka rekurencja może zawiesić lub zamknąć program. Jeśli potrzebujesz tysięcy poziomów, lepsza jest pętla.

Fibonacci naiwnie. Ciąg: `F(0) = 0`, `F(1) = 1`, `F(n) = F(n-1) + F(n-2)`. Zapis wprost z definicji jest bardzo prosty:

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

Jest jednak **bardzo wolny**: te same wartości liczone są od nowa wiele razy. Samo `fib(20)` robi aż 21891 wywołań, a czas rośnie wykładniczo (mniej więcej $O(2^n)$), więc `fib(40)` trwa już długo. Ćwiczenie 2 pokaże Ci to na liczbach.

Pamiętanie wyników (`lru_cache`). Wystarczy dopisać jedną linię nad definicją, a Python zapamięta wyniki i nie policzy tej samej wartości dwa razy. (Taka linia z `@` to *dekorator*. Na razie nie musisz rozumieć, jak działa, wystarczy ją wpisać.)

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(90))      # 2880067194370816120, natychmiast
```

Po dodaniu pamięci liczba obliczeń spada do O(n). Uwaga: przy `fib(2000)` i tak dostaniesz `RecursionError`, bo rekurencja jest zbyt głęboka. Wtedy pomaga wersja iteracyjna:

```
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Kiedy rekurencja jest dobra, a kiedy zła. Dobra jest tam, gdzie problem sam składa się z mniejszych takich samych problemów: drzewa (BST na jednych z kolejnych zajęć), sortowanie szybkie i przez scalanie, przeszukiwanie grafów (DFS), wieże Hanoi. Zła jest, gdy głębokość idzie w tysiące (np. `silnia(5000)`) albo gdy te same podproblemy są liczone wielokrotnie (Fibonacci bez pamięci). Na maturze często pada polecenie: *napisz wersję rekurencyjną i iteracyjną tego algorytmu*, więc ćwicz obie.

Ćwiczenia

Ćwiczenie 1 ★ *(plik `t5_cw1.py`)* Napisz **rekurencyjnie** funkcje `silnia(n)` i `suma_cyfr(n)`. Podpowiedź do drugiej: warunek stopu to `n < 10` (wynik to samo `n`), a krok to `n % 10 + suma_cyfr(n // 10)`. Wypisz `silnia(10)` i `suma_cyfr(98765)`.

Oczekiwany wynik:

```
3628800
35
```

Ćwiczenie 2 ★★ *(plik `t5_cw2.py`)* Napisz dwie wersje funkcji Fibonacciego: `fib(n)` (naiwną) i `fib2(n)` (z `@lru_cache(maxsize=None)`). W wersji naiwnej policz wszystkie wywołania w zmiennej `wywolania`, którą zwiększasz na początku funkcji. Dla funkcji, która ma zmieniać zmienną spoza siebie, tu **wyjątkowo** dopuszczamy `global wywolania`, bo to eksperyment, a nie normalny kod. W wersji z pamięcią liczbę rzeczywistych obliczeń podaje `fib2.cache_info().misses`. Wypisz wyniki w poniższej postaci.

Oczekiwany wynik:

```
fib(20) = 6765 wywołań: 21891
fib2(20) = 6765 wywołań: 21
fib2(90) = 2880067194370816120
```

Podpowiedź: kolejność w kodzie ma znaczenie. Wypisz `fib(20)` i licznik w jednym `print`, a `fib2(20)` wywołaj raz przed wypisaniem `cache_info()`.

Ćwiczenie 3 ★★★ *(plik `t5_cw3.py`)* Napisz rekurencyjnie cztery funkcje:

(a) `nwd(a, b)` algorytmem Euklidesa: `nwd(a, b) = nwd(b, a % b)`, a gdy `b == 0`, wynikiem jest `a`.

(b) `binarne(n)`, która zwraca napis z zapisem dwójkowym liczby (bez `bin`): dla `n < 2` wynik to `str(n)`, w przeciwnym razie `binarne(n // 2) + str(n % 2)`.

(c) `hanoi(n, z, na, pomoc)`, która **wypisuje** kolejne ruchy w *wieżach Hanoi* (przenieś `n` krążków ze słupka `z` na słupek `na`, używając `pomoc`). Rozwiązanie: (1) przenieś `n - 1` krążków ze `z` na `pomoc`, (2) wypisz ruch `z -> na`, (3) przenieś `n - 1` krążków z `pomoc` na `na`. Warunek stopu: `n == 0`, nic nie rób.

(d) `ruchy(n)`, czyli liczba ruchów potrzebnych dla `n` krążków: `ruchy(0) = 0`, `ruchy(n) = 2 * ruchy(n - 1) + 1`.

Wypisz `nwd(1071, 462)`, `binarne(202)`, wynik `hanoi(3, "A", "C", "B")` oraz `ruchy(10)` w podanej postaci.

Oczekiwany wynik:

```
21
11001010
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
Ruchy dla 10 krążków: 1023
```

Zadanie dodatkowe: łamanie szyfru Cezara (w domu)

To zadanie łączy napisy, funkcje, słowniki i `Counter` w jedno. **Nie ma tu oczekiwanych wyników** (poza jednym testem), bo sprawdzenie, czy tekst po odszyfrowaniu ma sens, jest częścią zadania. Zrób wszystkie podpunkty w jednym pliku `dodatkowe.py`.

Szyfr Cezara przesuwa każdą literę alfabetu o `k` miejsc (z zawinięciem po `z`). W plikach `tajne1.txt` i `tajne2.txt` są teksty po polsku, **bez polskich znaków diakrytycznych**, zapisane małymi literami. Szyfrowane są tylko litery `a` – `z`, a spacje, kropki i przecinki zostają bez zmian.

1. Napisz funkcję `szyfruj(tekst, k)`, która zwraca tekst zaszyfrowany kluczem `k`, oraz funkcję `odszyfruj(tekst, k)`, która go odszyfrowuje (podpowiedź: to szyfrowanie o `-k`). Znaki inne niż litery `a` – `z` przepisuj bez zmian. Sprawdź test: `szyfruj("ala ma kota", 3)` ma dać `dod pd nrwd`.
2. Wczytaj `tajne1.txt` i odszyfruj go, wiedząc, że **klucz wynosi 5**. Wypisz wynik na ekran.
3. W `tajne2.txt` jest tekst zaszyfrowany **nieznanym kluczem**. Znajdź go metodą analizy częstości: w długim polskim tekście najczęstszą literą jest zwykle `a`. Policz litery w zaszyfrowanym tekście (`Counter`), weź najczęstszą i wylicz, o ile została przesunięta względem litery `a` (`(ord(najczestsza) - ord("a")) % 26`). Wypisz klucz i odszyfrowany tekst.
4. Zapisz wynik do pliku `odszyfrowane.txt`: w pierwszej linii `Klucz: ...`, w następnych odszyfrowany tekst.
5. **Dla chętnych:** jeśli kiedyś założenie o literze `a` zawiedzie (inny tekst, inna najczęstsza litera), program powinien wypisać odszyfrowanie dla trzech najczęstszych liter zaszyfrowanego tekstu, żeby człowiek wybrał sensowne.

Wskazówki:

- Najpierw napisz i przetestuj `szyfruj` na krótkim słowie, dopiero potem bierz się za pliki.
- Do liczenia liter bierz tylko litery (`znak.isalpha()`), bez spacji i znaków interpunkcyjnych.
- Jeśli odszyfrowany tekst to bezsens, sprawdź, czy kierunek przesunięcia nie jest odwrócony.
- Na maturze za zadanie z kilkoma podpunktami dostaje się punkty za każdy z nich osobno, więc działający podpunkt 2 jest wart więcej niż niedokończone wszystko.

Ściągawka z zajęć

Potrzebuję...	Piszę...
funkcję zwracającą wynik	`def f(x):` + `return ...`
zwrócić dwie wartości i je odebrać	`return a, b` / `x, y = f(...)`
liczenie wystąpień w słowniku	`d[k] = d.get(k, 0) + 1`
grupowanie do list	`d.setdefault(k, []).append(x)`
część wspólna / suma / różnica zbiorów	`a & b` / `a | b` / `a - b`
usunięcie duplikatów z listy	`set(lista)`
lista z warunkiem	`[x for x in l if warunek]`
sortowanie wg własnego klucza	`sorted(l, key=lambda p: p[1])`
malejąco po liczbie, przy remisie rosnąco po napisie	`key=lambda p: (-p[1], p[0])`
najczęstsze elementy	`Counter(x).most_common(3)`
wszystkie permutacje / kombinacje / napisy z powtórzeniami	`permutations`, `combinations`, `product(..., repeat=n)`
NWD / NWW	`math.gcd(a, b)` / `math.lcm(a, b)`
ile elementów jest mniejszych / nie większych od x (lista posortowana)	`bisect_left(l, x)` / `bisect_right(l, x)`
rekurencja	warunek stopu + krok na mniejszym problemie
przyspieszenie rekurencji z powtarzającymi się wywołaniami	`@lru_cache(maxsize=None)`